Bab 6 Manajemen Sesi Anonim dan Keamanan Token QR

Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. **(C2 – Memahami)** menjelaskan kebutuhan sesi anonim serta membedakan sesi anonim, token QR, dan token sesi.
2. **(C3 – Menerapkan)** membuat token *opaque* yang dibangkitkan secara kriptografis serta menggunakannya dalam alur akses berbasis kode QR.
3. **(C4 – Menganalisis)** menganalisis risiko penebakan token, penyebaran QR, enumerasi sumber daya, dan kebocoran basis data.
4. **(C6 – Mencipta)** merancang mekanisme identifikasi pengguna anonim yang membatasi wewenang, masa hidup akses, dan dampak kebocoran token.

Orientasi Bab

Pada Bab 5, Anda telah mempelajari registrasi, autentikasi, dan otorisasi pengguna yang memiliki akun. Namun, tidak semua pengguna aplikasi perlu membuat akun. Dalam aplikasi *Kantin Multi-Tenant*, pelanggan seharusnya dapat memesan makanan cukup dengan memindai kode QR pada meja yang sedang ia gunakan.

Kemudahan itu membawa konsekuensi keamanan yang khas. Sistem tidak mengetahui *siapa* pelanggan, tetapi tetap harus mengetahui dari *meja mana* pesanan dibuat, berapa lama akses berlaku, dan bagaimana akses dicabut jika kode QR atau tautannya tersebar.

Bab ini membahas sesi anonim (*anonymous session*), token buram (*opaque token*), kode QR sebagai titik masuk, serta pertahanan berlapis terhadap penyalahgunaan token.

Peta bab. Seluruh potongan kode yang menyebutkan nama berkas diambil dari proyek pendamping *Kantin Multi-Tenant* (tag `pertemuan-06`). Seluruh angka pada contoh perhitungan bersifat **ilustratif**.

Prasyarat. Bab ini melanjutkan Bab 4 dan Bab 5. Anda perlu memahami isolasi tenant, *middleware*, *policy*, serta perbedaan autentikasi dan otorisasi sebelum menerapkan kode pada bab ini.

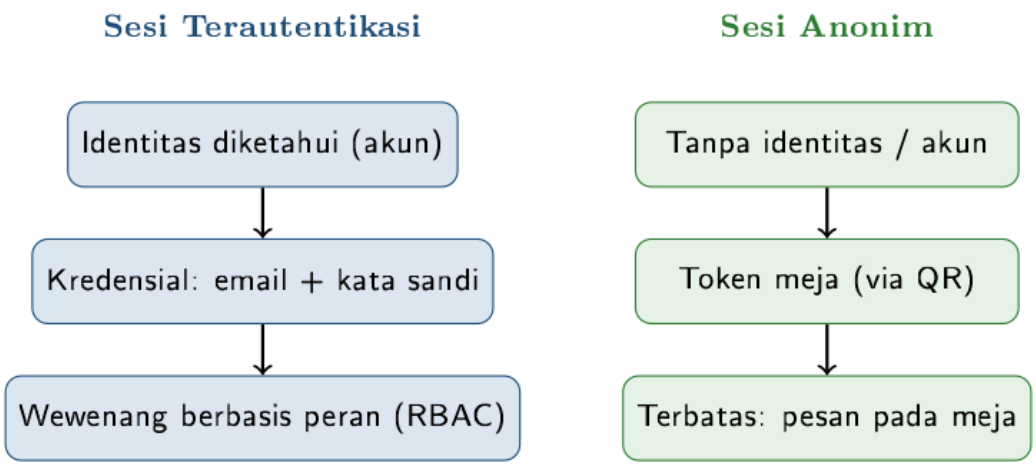
6.1 Mengapa Aplikasi Memerlukan Sesi Anonim

6.1.1 Akses Layanan Tanpa Akun

Sesi adalah mekanisme yang memungkinkan server mempertahankan konteks pengguna di antara beberapa permintaan HTTP. Pada aplikasi berbasis akun, konteks itu dikaitkan dengan identitas pengguna — misalnya pengelola kantin, operator tenant, atau kasir yang telah masuk.

Namun, beberapa layanan perlu mengurangi hambatan penggunaan. Pelanggan yang sudah duduk di meja tidak perlu membuat akun hanya untuk melihat menu dan memesan. Dalam situasi ini, aplikasi memakai **sesi anonim**.

Perhatikan pergeseran penting berikut. Sesi anonim tidak mengidentifikasi **orang**; sesi anonim mengidentifikasi **konteks layanan**, yaitu meja yang sedang digunakan. Karena tidak ada identitas yang diverifikasi, wewenang sesi anonim harus dibatasi secara ketat.



Gambar 6.1. Perbandingan sesi terautentikasi dan sesi anonim.

Sumber: diadaptasi penulis, ilustratif.

Aspek	Sesi terautentikasi	Sesi anonim
Dasar pengikatan	Identitas pengguna	Konteks layanan, yaitu meja dan kantin
Bukti akses	Kredensial akun, misalnya email dan kata sandi	Kepemilikan token akses melalui QR
Wewenang	Berdasarkan peran (RBAC)	Sangat terbatas pada tindakan yang relevan dengan meja
Masa hidup	Sampai keluar, kedaluwarsa, atau dicabut	Sampai kedaluwarsa, ditutup, atau tokennya dicabut
Contoh pengguna	Pengelola kantin, operator tenant, kasir	Pelanggan yang memesan dari meja

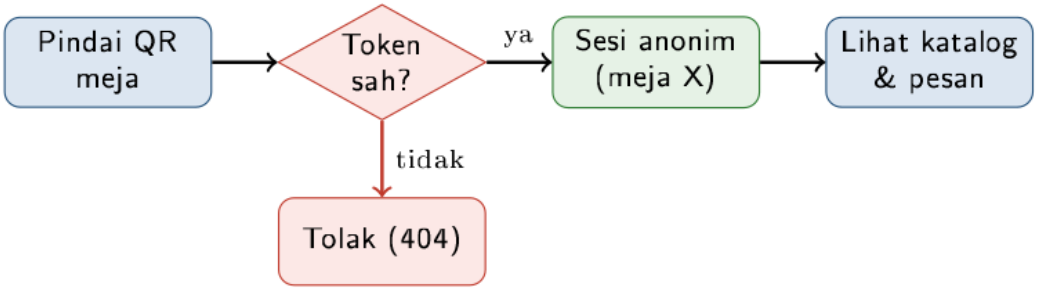
Tabel 6.1. Perbandingan sesi terautentikasi dan sesi anonim.

Perbedaan yang paling penting terletak pada tingkat kepercayaan. Sesi terautentikasi dapat memberikan akses sesuai peran pengguna karena sistem telah memverifikasi identitasnya. Sesi anonim hanya boleh memberi akses yang benar-benar diperlukan untuk menyelesaikan satu layanan.

Peringatan. Token QR **bukan** bukti identitas pelanggan. Token tersebut adalah *bearer credential*: siapa pun yang memegang tautan atau hasil pemindaian yang masih berlaku dapat mencoba menggunakannya. Karena itu, token QR tidak boleh memberi wewenang administrasi, membuka data sensitif, atau mengubah konfigurasi tenant.

6.1.2 Studi Kasus Pembuka: Pemesanan dari Meja Kantin

Bayangkan pelanggan memindai kode QR pada sebuah meja. Peramban membuka URL aplikasi. Server mencari token tersebut, memastikan token masih aktif, menentukan meja dan kantin yang terkait, lalu membentuk sesi anonim. Pelanggan kemudian dapat melihat katalog dan memesan.



Gambar 6.2. Alur pemesanan pelanggan tanpa registrasi akun.

Sumber: diadaptasi penulis, ilustratif.

Sistem pada contoh ini mengenali **meja**, bukan orang yang memegang ponsel. Konsekuensinya, setiap pesanan harus selalu diperiksa terhadap sesi anonim yang aktif dan terhadap meja yang terikat pada sesi tersebut — bukan terhadap nomor meja yang dikirim dari formulir.

Alur ini mengurangi hambatan pemesanan, tetapi tidak cocok untuk semua kondisi. Bila aplikasi perlu menangani pembayaran pascabayar perorangan, riwayat pesanan pribadi, atau program loyalitas, sistem tetap memerlukan akun atau mekanisme identifikasi tambahan.

6.2 Konteks Meja, Token QR, dan Token Sesi

Sebelum membahas kode, kita perlu membedakan dua token yang fungsinya berbeda dan sering tertukar.

- **Token QR** mengarahkan pelanggan ke konteks meja tertentu. Token ini tertanam dalam URL yang disandikan oleh kode QR. Ia adalah *titik masuk*, dipakai satu kali per pemindaian.
- **Token sesi** membuktikan bahwa peramban pelanggan memiliki sesi anonim yang masih aktif. Ia dipakai *berulang kali* selama pelanggan menelusuri katalog dan mengirim pesanan.

Pemisahan ini bukan sekadar kerapian. Kedua token memiliki masa hidup, tempat penyimpanan, dan cara pencabutan yang berbeda. Merotasi token QR sebuah meja tidak boleh menghentikan pelanggan yang sedang makan di meja itu, dan sebaliknya, berakhirnya sesi seorang pelanggan tidak boleh membuat meja tersebut tidak dapat dipakai pelanggan berikutnya.

6.2.1 Model Data pada Proyek Pendamping

Proyek pendamping memakai tiga tabel yang terpisah, bukan dua. Berkasnya adalah migration `create_table_session_tables`.

Kode 6.1 — Migration create_table_session_tables (potongan)

```php
Schema::create('dining_tables', function (Blueprint $table): void {
    $table->id();
    $table->foreignId('canteen_id')->constrained()->restrictOnDelete();
    $table->string('code', 30);
    $table->string('label', 60);
    $table->string('zone', 60)->nullable();
    $table->string('status', 20)->default('active'); // active|inactive
    $table->timestamps(6);

    $table->unique(['canteen_id', 'code']);
    $table->unique(['id', 'canteen_id']);
});

Schema::create('table_qr_tokens', function (Blueprint $table): void {
    $table->id();
```

```
    $table->foreignId('dining_table_id')->constrained()->cascadeOnDelete();
    $table->binary('token_hash', 32);
    $table->string('status', 20)->default('active'); // active|revoked|expired
    $table->timestampTz('issued_at', 6)->nullable();
    $table->timestampTz('expires_at', 6)->nullable();
    $table->timestampTz('revoked_at', 6)->nullable();
    $table->timestamps(6);

    $table->unique('token_hash');
    $table->index(['dining_table_id', 'status']);
});

Schema::create('customer_sessions', function (Blueprint $table): void {
    $table->ulid('id')->primary();
    $table->foreignId('canteen_id')->constrained()->restrictOnDelete();
    $table->foreignId('dining_table_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('qr_token_id')->nullable()
        ->constrained('table_qr_tokens')->nullOnDelete();
    $table->binary('session_token_hash', 32);
    $table->string('status', 20)->default('active'); // active|expired|closed
    $table->timestampTz('expires_at', 6)->nullable();
    $table->timestamps(6);

    $table->unique('session_token_hash');
    $table->index(['canteen_id', 'status']);
});
```

Perhatikan empat keputusan desain pada Kode 6.1.

Keputusan	Alasan
Token QR disimpan pada tabel sendiri, bukan sebagai kolom meja	Satu meja dapat memiliki riwayat banyak token. Rotasi menandai token lama sebagai `revoked` dan menerbitkan baris baru, sehingga jejak rotasi tetap dapat diaudit.
`token_hash` bertipe `binary(32)`, bukan teks	Kolom menyimpan hasil SHA-256 mentah (32 bita), bukan bentuk heksadesimalnya. Pembandingan menjadi separuh lebih pendek dan indeksnya lebih ringkas.
Kunci utama `customer_sessions` berupa ULID	Pengenal sesi tidak berurutan, sehingga tidak dapat ditebak atau dipakai memperkirakan jumlah pelanggan.
Sesi terikat pada `canteen_id` dan `dining_table_id`	Konteks kantin tetap diketahui meskipun meja kemudian dihapus. Kolom meja memakai `nullOnDelete()`, sedangkan kolom kantin memakai `restrictOnDelete()`.

Tabel 6.2. Keputusan desain pada skema meja, token QR, dan sesi pelanggan.

Server tidak pernah menyimpan token mentah. Baik token QR maupun token sesi hanya tersimpan sebagai *hash*. Jika basis data bocor, token mentah tidak ikut bocor: penyerang memperoleh *hash* yang tidak dapat langsung dikirim sebagai kredensial.

> **Catatan.** Untuk kata sandi, algoritme *hash* yang lambat seperti bcrypt atau Argon2 memang disarankan justru karena lambatnya menyulitkan penebakan. Untuk token acak berentropi tinggi, SHA-256 yang cepat sudah memadai: tidak ada “kata sandi umum” yang dapat dicoba, sehingga penyerang harus menjelajahi seluruh ruang token. Perbedaan ini sering tertukar di kelas.

6.2.2 Batas Wewenang Sesi Anonim

Sesi anonim sebaiknya hanya mampu melakukan tindakan berikut:

- melihat katalog yang memang tersedia untuk kantin dan meja terkait;
- menambah, mengubah, atau menghapus item pesanan selama pesanan masih berada dalam konteks meja yang sama;
- mengirim pesanan sesuai aturan bisnis aplikasi.

Sesi anonim **tidak** seharusnya mampu melakukan tindakan berikut:

- melihat daftar seluruh meja pada sebuah kantin;
- mengakses pesanan dari meja lain;
- membuka data pribadi pelanggan atau pegawai;
- mengubah menu, harga, stok, tenant, atau konfigurasi aplikasi;
- menjalankan fungsi administrasi apa pun.

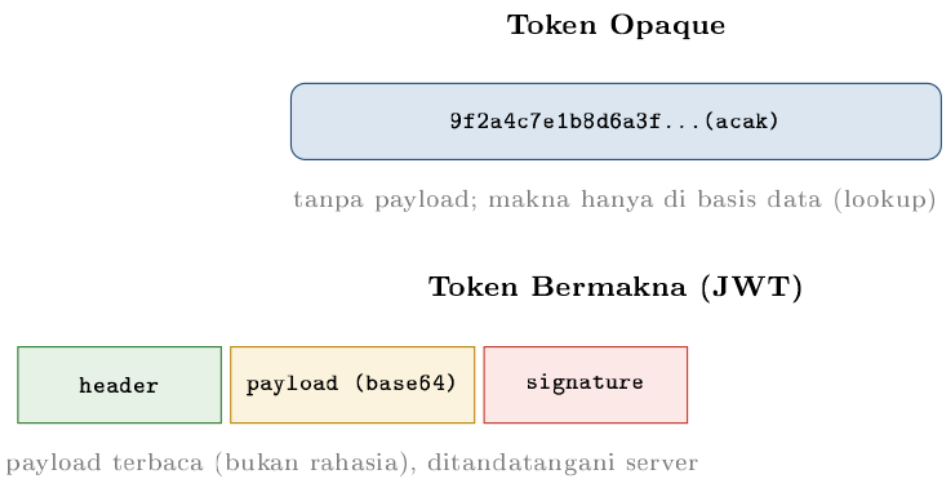
Pembatasan ini adalah penerapan prinsip *least privilege* yang telah Anda kenal dari Bab 5. Perbedaannya, di sini prinsip itu diterapkan pada subjek yang identitasnya sama sekali tidak diketahui — sehingga batasannya harus lebih ketat, bukan lebih longgar.

6.3 Token Opaque dan Siklus Hidupnya

6.3.1 Apa Itu Token Opaque?

Token buram (*opaque token*) adalah untai karakter acak yang tidak membawa makna bisnis di dalam dirinya. Server memperoleh arti token dengan mencarinya pada penyimpanan sisi server.

Sebaliknya, *JSON Web Token* (JWT) membawa klaim atau *payload* yang dapat didekodekan. JWT dapat ditandatangani agar integritasnya terverifikasi, tetapi isi *payload* tidak otomatis terenkripsi — siapa pun yang memegangnya dapat membacanya.



Gambar 6.3. Anatomi token opaque dan token bermakna (JWT).

Sumber: diadaptasi penulis, ilustratif.

Aspek	Token opaque	JWT
Isi token	Untai acak tanpa makna bisnis	Membawa klaim yang dapat didekodekan

Cara validasi	Pencarian ke penyimpanan server	Verifikasi tanda tangan dan pemeriksaan klaim
Pencabutan	Langsung, melalui perubahan data server	Memerlukan strategi tambahan: *denylist*, versi token, atau masa berlaku pendek
Informasi yang tampak	Tidak mengungkap data konteks	*Payload* dapat dibaca bila tidak dienkripsi
Kebutuhan penyimpanan	Ya, untuk *lookup*	Dapat lebih *stateless*, tetapi pencabutan dan audit tetap butuh penyimpanan
Kesesuaian pada kasus meja	Baik: akses perlu mudah dicabut dan terikat data server	Dapat dipakai, tetapi menambah kompleksitas yang tidak diperlukan

Tabel 6.3. Perbandingan token opaque dan JWT.

Untuk akses meja kantin, token *opaque* lebih sesuai karena server perlu dapat mencabut akses dengan segera — saat QR diganti, meja dinonaktifkan, atau muncul indikasi penyalahgunaan. Dengan JWT, pencabutan sebelum kedaluwarsa justru menuntut penyimpanan sisi server juga, sehingga keunggulan *stateless*-nya hilang.

6.3.2 Entropi Token dan Sumber Keacakan

Keamanan token acak bergantung pada **entropi**, yaitu tingkat ketakterdugaan nilainya. Jika token tersusun dari *L* karakter dengan *N* kemungkinan simbol pada setiap posisi, entropinya adalah:

$$H = L \cdot \log_2 N$$

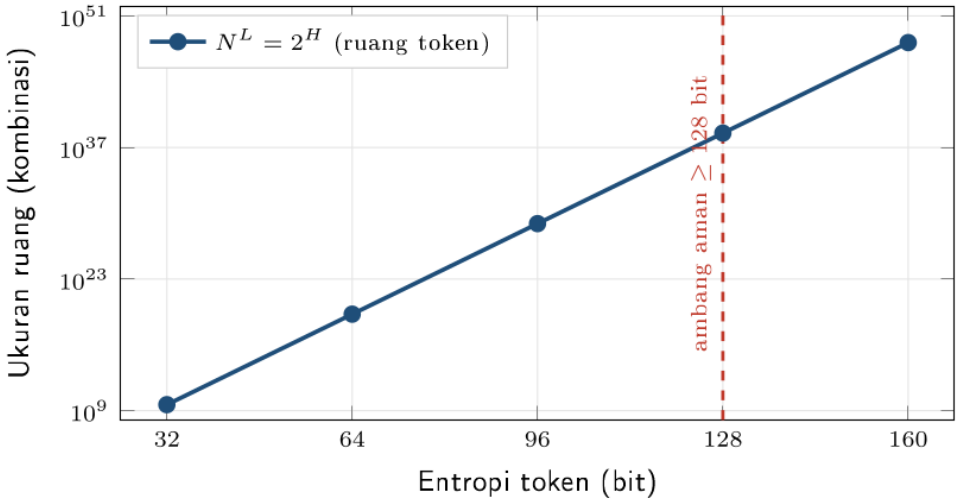
(Persamaan 6.1)

dengan *H* adalah entropi token dalam bit, *L* adalah jumlah karakter token, dan *N* adalah jumlah simbol yang mungkin pada setiap posisi.

Persamaan 6.1 hanya berlaku apabila setiap posisi dipilih secara merata dan independen dari sumber acak yang aman secara kriptografis. Jika token dibentuk dari nama meja, waktu, atau nomor urut, panjangnya boleh sama tetapi entropinya jauh lebih kecil.

Bentuk token	Panjang	Entropi	Interpretasi pembelajaran
Heksadesimal	8 karakter	32 bit	Terlalu kecil untuk token akses publik
Heksadesimal	16 karakter	64 bit	Lebih baik, tetapi belum memadai untuk akses bernilai
Heksadesimal	24 karakter	96 bit	Kuat untuk banyak penggunaan
Heksadesimal	32 karakter	128 bit	Ambang yang lazim dianggap aman
Base64url	43 karakter	256 bit	Pilihan proyek pendamping (32 bita acak)

Tabel 6.4. Entropi token menurut bentuk dan panjangnya; data ilustratif.



Gambar 6.4. Pertumbuhan ukuran ruang token terhadap entropi (skala logaritmik).

Sumber: diadaptasi penulis, data ilustratif.

Catatan. Baris terakhir Tabel 6.4 memerlukan kehati-hatian. Menerapkan Persamaan 6.1 pada 43 karakter base64url memberi 43 × 6 = 258 bit, sedangkan entropi sebenarnya hanya 256 bit — yaitu sebanyak bita acak yang dibangkitkan. Selisih dua bit itu muncul karena 256 bukan kelipatan enam, sehingga karakter terakhir tidak membawa enam bit penuh. Pelajarannya: entropi ditentukan oleh **sumber acak**, bukan oleh panjang teks hasil pengodean. Persamaan 6.1 memberi batas atas, bukan nilai pasti, ketika pengodean tidak sejajar dengan batas bit.

6.3.3 Membuat Token yang Aman

Pada PHP, gunakan `random_bytes()` yang mengambil bita acak dari sumber yang sesuai untuk kebutuhan kriptografis. Jangan memakai `rand()` atau `mt_rand()` untuk token akses, karena keduanya tidak dirancang sebagai sumber acak kriptografis dan keluarannya dapat diperkirakan.

Proyek pendamping mengemas pembangkitan dan *hashing* token dalam satu kelas kecil. Berkasnya adalah `app/Support/Tokens/OpaqueToken.php`.

Kode 6.2 — app/Support/Tokens/OpaqueToken.php

```php
<?php

namespace App\Support\Tokens;

use InvalidArgumentException;

final class OpaqueToken
{
    /** @return array{plain: string, hash: string} */
    public static function issue(int $bytes = 32): array
    {
        if ($bytes < 16) {
            throw new InvalidArgumentException('Token minimal 128 bit (16 byte).');
        }

        $plain = rtrim(
            strtr(base64_encode(random_bytes($bytes)), '+/', '-_'),
            '='
        );
```

```
        return ['plain' => $plain, 'hash' => hash('sha256', $plain, true)];
    }

    public static function hash(string $plain): string
    {
        return hash('sha256', $plain, true);
    }
}
```

Baca kode tersebut sebagai empat keputusan.

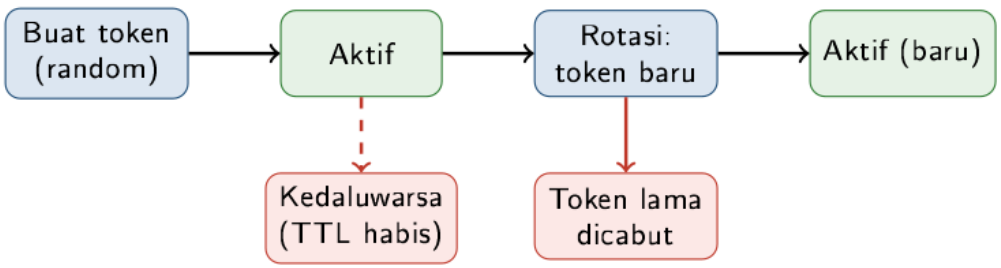
- `random_bytes(32)` menghasilkan 32 bita acak kriptografis, yaitu entropi 256 bit.
- `strtr(..., '+/', '-_')` mengubah alfabet base64 standar menjadi **base64url**, sehingga token aman disisipkan ke dalam URL tanpa pengodean tambahan.
- `rtrim(..., '=')` membuang karakter isian yang tidak membawa informasi dan justru merepotkan pada URL.
- Argumen ketiga `true` pada `hash()` meminta keluaran biner mentah 32 bita, bukan 64 karakter heksadesimal — sesuai kolom `binary(32)` pada Kode 6.1.

Perhatikan pula bahwa metode `issue()` mengembalikan **dua** nilai sekaligus. Nilai `plain` diberikan sekali kepada pemanggil untuk dikirim ke klien, sedangkan `hash` disimpan ke basis data. Setelah pemanggilan itu, tidak ada cara memperoleh kembali token mentahnya — persis seperti kata sandi.

Peringatan. Pemeriksaan `$bytes < 16` bukan hiasan. Tanpa itu, seorang pengembang dapat memanggil `issue(4)` untuk "menyederhanakan pengujian" dan tanpa sadar menurunkan entropi token menjadi 32 bit. Batas minimum yang ditegakkan kode jauh lebih andal daripada batas minimum yang hanya ditulis di dokumen.

6.3.4 Penerbitan, Rotasi, dan Pencabutan

Token memiliki siklus hidup: dibuat, dipakai selama masih sah, lalu kedaluwarsa, dicabut, atau dirotasi.



Gambar 6.5. Siklus hidup token QR: penerbitan, kedaluwarsa, rotasi, dan pencabutan.

Sumber: diadaptasi penulis, ilustratif.

- **Kedaluwarsa** membatasi lama penggunaan token melalui kolom `expires_at`.
- **Pencabutan** menghentikan token sebelum masa berlakunya berakhir, dengan mengubah `status` menjadi `revoked` dan mengisi `revoked_at`.
- **Rotasi** menerbitkan token baru sekaligus mencabut yang lama.

Pada proyek pendamping, ketiganya ditangani satu layanan. Berkasnya adalah `app/Modules/Admin/Services/QrTokenService.php`.

Kode 6.3 — app/Modules/Admin/Services/QrTokenService.php

```php
public function issue(DiningTable $table, ?CarbonInterface $expiresAt = null): string
{
    return DB::transaction(function () use ($table, $expiresAt): string {
        $locked = DiningTable::query()->lockForUpdate()->findOrFail($table->id);

        TableQrToken::query()
            ->where('dining_table_id', $locked->id)
            ->active()
            ->update(['status' => 'revoked', 'revoked_at' => now()]);

        $token = OpaqueToken::issue(32);
        $qr = new TableQrToken;
        $qr->forceFill([
            'dining_table_id' => $locked->id,
            'token_hash' => $token['hash'],
            'status' => 'active',
            'issued_at' => now(),
            'expires_at' => $expiresAt,
            'revoked_at' => null,
        ])->save();

        $this->audit->record('table_qr_token', $qr->id, 'issued',
            null, ['dining_table_id' => $locked->id], null, $locked->canteen_id);

        return $token['plain'];
    });
}

/** Rotasi = terbitkan token baru (otomatis mencabut yang aktif). */
public function rotate(DiningTable $table): string
{
    return $this->issue($table);
}
```

Perhatikan bahwa `rotate()` hanya memanggil `issue()`. Ini bukan kemalasan, melainkan penyederhanaan yang disengaja: menerbitkan token baru **selalu** mencabut token aktif sebelumnya, sehingga tidak mungkin ada dua token aktif untuk satu meja. Dengan begitu, tidak ada dua jalur kode yang harus dijaga tetap konsisten.

Ketiga langkah — mengunci baris meja, mencabut token lama, menerbitkan token baru — berada dalam satu transaksi. `lockForUpdate()` mencegah dua permintaan rotasi yang bersamaan sama-sama menerbitkan token aktif.

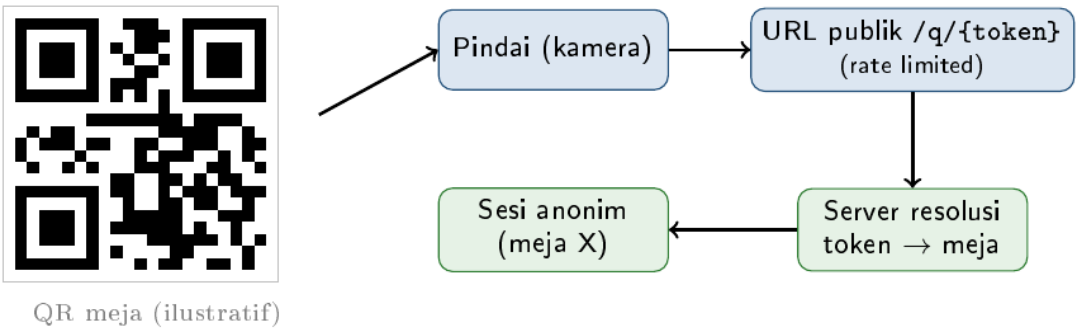
Token lama tidak dihapus, melainkan ditandai `revoked`. Riwayat penerbitan tetap tersimpan sehingga pertanyaan seperti “kapan token meja ini terakhir dirotasi dan siapa yang melakukannya” dapat dijawab dari basis data dan *audit trail*.

> **Tip.** Rotasi memiliki dampak operasional: QR yang telah dicetak dan menyandikan token lama harus diganti. Karena itu, rotasi rutin cocok untuk QR yang ditampilkan pada layar atau dicetak murah, sedangkan QR yang dicetak permanen sebaiknya dirotasi hanya ketika ada indikasi penyalahgunaan. Pilihan ini dibahas kembali pada Latihan nomor 12.

6.4 QR sebagai Titik Masuk yang Terkendali

6.4.1 QR, URL, dan Pertanyaan tentang Signed URL

Kode QR (*Quick Response code*) adalah barkode dua dimensi yang dapat menyandikan teks, termasuk URL. Pada aplikasi ini, QR mengarahkan pelanggan ke titik masuk meja.



Gambar 6.6. Kode QR sebagai titik masuk menuju sesi anonim.

Sumber: diadaptasi penulis, ilustratif.

Laravel menyediakan *signed URL*, yaitu URL yang diberi tanda tangan sehingga server dapat mendeteksi perubahan pada parameternya. Pertanyaan yang wajar muncul: mengapa proyek pendamping **tidak** memakainya pada rute pemindaian QR?

Jawabannya terletak pada apa yang sebenarnya dilindungi masing-masing mekanisme. Tanda tangan URL melindungi **integritas** parameter — ia mencegah orang mengubah nilai pada URL yang ia terima. Tanda tangan tidak membuat token menjadi rahasia dan tidak mencegah seseorang menyalin URL yang sah.

Pada rute ini, satu-satunya parameter adalah token itu sendiri, dan token sudah dilindungi oleh entropinya: mengubahnya berarti menebak token lain sebesar 256 bit. Menambahkan tanda tangan tidak menambah perlindungan, tetapi menambah masalah operasional yang nyata.

> **Peringatan.** *Signed URL* sementara dan QR cetak permanen sulit dipadukan. URL bertanda tangan sementara berlaku hingga waktu tertentu; setelah lewat, seluruh QR yang menyandikannya menjadi tidak berguna dan harus dicetak ulang. Karena itu, pilihan proyek pendamping adalah URL publik sederhana berisi token berentropi tinggi, dilindungi pembatasan laju dan pencabutan sisi server.

Rute pemindaian pada proyek pendamping karena itu sangat ringkas. Berkasnya adalah `routes/web.php`.

Kode 6.4 — routes/web.php (potongan rute pemindaian QR)

```php
// Entry point pelanggan via QR meja.
// Token opaque di URL; rate limited; canteen ditentukan dari token.
Route::get('/q/{token}', ResolveTableQrController::class)
    ->middleware('throttle:qr-scan')
    ->name('customer.scan');
```

Pembatas laju `qr-scan` didefinisikan pada berkas `app/Providers/AppServiceProvider.php`.

Kode 6.5 — app/Providers/AppServiceProvider.php (potongan pembatas laju)

```php
RateLimiter::for(
    'qr-scan',
    fn (Request $request): Limit => Limit::perMinute(30)
        ->by($request->ip() ?? 'unknown')
);
```

Perhatikan juga bentuk URL-nya: `/q/{token}`, bukan `/kantin/{canteen}/meja/{id}`. URL itu tidak menyebut kantin maupun nomor meja. Kantin dan meja **ditentukan dari token**, bukan dari

masukan klien. Dengan begitu tidak ada parameter yang dapat dimanipulasi, dan tidak ada pengenal berurutan yang dapat dienumerasi.

> **Catatan.** Pembatasan laju berbasis alamat IP memiliki dua kelemahan yang saling berlawanan. Pelanggan sah yang berbagi satu jaringan Wi-Fi kantin akan terhitung sebagai satu klien, sedangkan penyerang dapat memakai banyak alamat IP. Karena itu, ambang 30 permintaan per menit dipilih cukup longgar untuk penggunaan normal dan tetap menghambat percobaan otomatis. Pembatasan laju adalah satu lapis pertahanan, bukan satu-satunya.

6.4.2 Menyelesaikan Pemindaian dan Membentuk Sesi

Ketika pelanggan memindai QR, server melakukan rangkaian pemeriksaan sebelum membentuk sesi. Berkasnya adalah `app/Modules/Ordering/Services/ResolveTableScan.php`.

Kode 6.6 — app/Modules/Ordering/Services/ResolveTableScan.php

```php
/** @return array{session: CustomerSession, plain: string, canteen: Canteen}|null */
public function resolve(string $plainToken): ?array
{
    $qr = TableQrToken::query()
        ->where('token_hash', OpaqueToken::hash($plainToken))
        ->with('diningTable.canteen')
        ->first();

    if ($qr === null || ! $qr->isActive()) {
        return null;
    }

    $table = $qr->diningTable;
    if ($table === null || ! $table->isActive()) {
        return null;
    }

    $canteen = $table->canteen;
    if (! $canteen instanceof Canteen || $canteen->status !== 'active') {
        return null;
    }

    $sessionToken = OpaqueToken::issue(32);
    $session = new CustomerSession;
    $session->forceFill([
        'canteen_id' => $canteen->id,
        'dining_table_id' => $table->id,
        'qr_token_id' => $qr->id,
        'session_token_hash' => $sessionToken['hash'],
        'status' => 'active',
        'expires_at' => now()->addHours(4),
    ])->save();

    return [
        'session' => $session,
        'plain' => $sessionToken['plain'],
        'canteen' => $canteen,
    ];
}
```

Layanan ini memeriksa empat hal berurutan sebelum menerbitkan sesi:

1. *hash* token ditemukan pada tabel `table_qr_tokens`;
2. token berstatus aktif, belum dicabut, dan belum kedaluwarsa;
3. meja yang terkait masih berstatus aktif;

4. kantin pemilik meja masih berstatus aktif.

Perhatikan bahwa seluruh kegagalan mengembalikan `null` yang sama, tanpa membedakan penyebabnya. Ini disengaja: pesan galat yang spesifik akan memberi tahu penyerang apakah sebuah token pernah ada, sudah dicabut, atau mejanya dinonaktifkan. Informasi itu tidak berguna bagi pelanggan sah, tetapi berguna bagi penyerang.

Perhatikan juga bahwa kueri mencari `token_hash`, bukan token mentah. Token dari URL langsung di-*hash*, lalu *hash* itulah yang dibandingkan dengan kolom basis data.

6.4.3 Menyimpan Token Sesi pada Cookie yang Aman

Setelah sesi terbentuk, token sesi mentah harus dikirim ke peramban pelanggan. Berkasnya adalah `app/Http/Controllers/Customer/ResolveTableQrController.php`.

Kode 6.7 — app/Http/Controllers/Customer/ResolveTableQrController.php

```php
public function __invoke(Request $request, string $token): RedirectResponse
{
    $result = $this->resolver->resolve($token);
    abort_if($result === null, 404);

    $cookie = cookie(
        name: 'customer_session',
        value: $result['plain'],
        minutes: 240,
        path: '/',
        domain: null,
        secure: $request->isSecure() || app()->isProduction(),
        httpOnly: true,
        raw: false,
        sameSite: 'lax',
    );

    return redirect()
        ->route('customer.home', ['canteen' => $result['canteen']->slug])
        ->withCookie($cookie);
}
```

Setiap atribut *cookie* pada Kode 6.7 memiliki alasan keamanan tersendiri.

Atribut	Nilai	Alasan
`httpOnly`	`true`	Skrip di halaman tidak dapat membaca *cookie*, sehingga celah XSS tidak langsung membocorkan token sesi.
`secure`	aktif pada HTTPS dan produksi	*Cookie* tidak dikirim melalui koneksi tanpa enkripsi.
`sameSite`	`lax`	*Cookie* tidak ikut terkirim pada permintaan lintas situs, sehingga menutup sebagian besar jalur CSRF.
`minutes`	240	Selaras dengan `expires_at` sesi selama 4 jam pada Kode 6.6.
`path`	`/`	*Cookie* berlaku pada seluruh jalur aplikasi pelanggan.

Tabel 6.5. Atribut cookie sesi pelanggan dan alasannya.

Perhatikan pula tujuan pengalihannya: `route('customer.home', ['canteen' => $result['canteen']->slug])`. Nilai `canteen` berasal dari hasil penelusuran token di basis data, **bukan** dari parameter yang dikirim klien. Pola ini penting dan berulang sepanjang buku: konteks selalu ditentukan sisi server dari data tepercaya.

Peringatan. Meskipun *cookie* berumur 240 menit sejajar dengan `expires_at` sesi, keduanya bukan mekanisme yang sama. *Cookie* berada di sisi klien dan dapat dimanipulasi atau disalin; `expires_at` berada di basis data dan tidak dapat disentuh klien. Kedaluwarsa yang mengikat adalah yang di basis data — dan aplikasi harus memeriksanya pada setiap permintaan penting, bukan mengandalkan peramban membuang *cookie* tepat waktu.

6.4.4 Memvalidasi Sesi pada Setiap Permintaan Penting

Keamanan tidak berhenti saat pelanggan masuk ke katalog. Ketika pelanggan membuat pesanan, aplikasi harus memvalidasi ulang token sesi. Pola *middleware*-nya adalah sebagai berikut.

Kode 6.8 — Contoh generik: middleware validasi sesi pelanggan

```php
public function handle(Request $request, Closure $next): Response
{
    $plain = $request->cookie('customer_session');
    abort_unless(is_string($plain) && $plain !== '', 403);

    $session = CustomerSession::query()
        ->where('session_token_hash', OpaqueToken::hash($plain))
        ->first();

    abort_unless($session !== null && $session->isActive(), 403);

    $request->attributes->set('customerSession', $session);

    return $next($request);
}
```

Metode `isActive()` pada model memusatkan aturan kedaluwarsa sehingga tidak diulang di banyak tempat. Berkasnya adalah `app/Models/CustomerSession.php`.

Kode 6.9 — app/Models/CustomerSession.php (potongan)

```php
public function isActive(): bool
{
    return $this->status === 'active'
        && ($this->expires_at === null || $this->expires_at->isFuture());
}
```

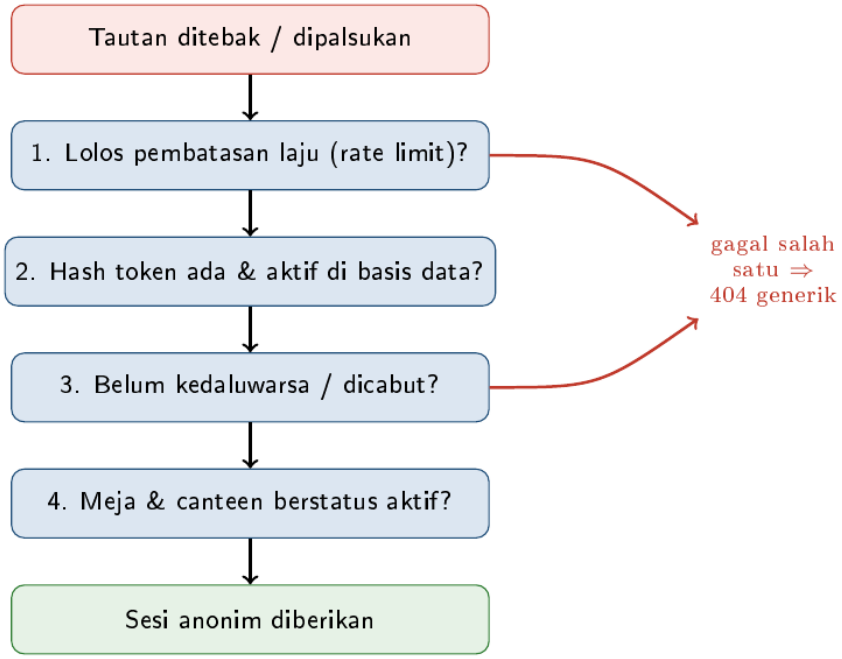
Dengan pola ini, aplikasi tidak pernah mempercayai nomor meja yang dikirim dari formulir pelanggan. Konteks meja diambil dari objek sesi yang telah divalidasi di sisi server, yaitu dari `$session->dining_table_id`.

Catatan. Sesi yang telah melewati `expires_at` sebaiknya dibersihkan secara berkala melalui pekerjaan terjadwal, agar tabel tidak tumbuh tanpa batas. Namun pembersihan **tidak menggantikan** validasi saat permintaan masuk. Server tetap harus menolak sesi kedaluwarsa meskipun barisnya belum sempat dihapus — jika tidak, keamanan aplikasi bergantung pada apakah sebuah *cron job* berjalan tepat waktu.

[PERLU VERIFIKASI: apakah proyek pendamping telah memiliki middleware validasi sesi pelanggan dan pekerjaan pembersihan sesi kedaluwarsa pada tag pertemuan-06, atau keduanya diperkenalkan pada modul berikutnya]

6.5 Pertahanan Berlapis dan Batasannya

6.5.1 Ancaman Utama dan Pertahanannya



Gambar 6.7. Empat lapis pemeriksaan sebelum sesi anonim diberikan.

Sumber: diadaptasi penulis, ilustratif.

Ancaman	Cara terjadi	Pertahanan pada proyek pendamping
Penebakan token	Penyerang mencoba banyak token secara otomatis	Entropi 256 bit, pembatasan laju 30 permintaan per menit
Enumerasi sumber daya	Penyerang mencoba pengenal berurutan	URL tidak memuat ID meja; ULID pada sesi; token acak
Pengintaian melalui pesan galat	Penyerang membaca beda respons untuk menyimpulkan keberadaan data	Seluruh kegagalan menghasilkan 404 generik
Penyebaran QR atau tautan	QR difoto atau URL diteruskan ke orang lain	TTL sesi 4 jam, rotasi token, pencabutan, wewenang terbatas
Kebocoran basis data	Salinan basis data jatuh ke pihak lain	Hanya *hash* token yang tersimpan; token mentah tidak dapat dipulihkan
Pencurian token lewat XSS	Skrip berbahaya membaca token pada klien	*Cookie* `httpOnly` sehingga tidak terbaca skrip halaman
Sesi kedaluwarsa dipakai ulang	Token sesi lama dikirim kembali	Pemeriksaan `status` dan `expires_at` pada setiap permintaan penting

Tabel 6.6. Ancaman penyalahgunaan dan mekanisme pencegahannya.

6.5.2 Mengapa Token Berentropi Tinggi Sulit Ditebak?

Jika terdapat *A* token aktif dan ruang kemungkinan token sebesar 2 pangkat *H*, peluang menebak salah satu token aktif dalam satu percobaan dapat didekati sebagai:

$$p_{\text{tebak}} \approx \frac{A}{N^L} = \frac{A}{2^H} \qquad \text{(Persamaan 6.2)}$$

dengan *p* adalah peluang keberhasilan dalam satu percobaan, *A* adalah jumlah token yang aktif, dan *H* adalah entropi token dalam bit.

Contoh 6.1. Misalkan sebuah kantin memiliki 1.000 token aktif dan setiap token berentropi 128 bit. Peluang keberhasilan satu percobaan adalah $1.000 / 2^{128} \approx 2{,}9 \times 10^{-36}$. Untuk token 256 bit seperti pada proyek pendamping, peluangnya menjadi $1.000 / 2^{256} \approx 8{,}6 \times 10^{-75}$.

Angka tersebut sangat kecil. Namun, entropi tinggi bukan alasan mengabaikan pertahanan lain. Yang dilindungi entropi hanyalah penebakan; ia tidak melindungi terhadap token yang **bocor** karena difoto, diteruskan, atau tersalin dari riwayat peramban.

Bandingkan dengan token 32 bit. Ruangnya hanya sekitar 4,3 miliar kemungkinan. Berapa lama token semacam itu dapat ditemukan bergantung pada tiga hal: jumlah token aktif, laju percobaan yang dapat dilakukan penyerang, dan ada tidaknya pembatasan laju.

Contoh 6.2. Misalkan token hanya berentropi 32 bit, terdapat 1.000 token aktif, dan sistem membatasi 30 percobaan per menit per alamat IP. Peluang per percobaan adalah $1.000 / 2^{32} \approx 2{,}3 \times 10^{-7}$. Untuk mencapai peluang kumulatif sekitar 50%, penyerang memerlukan kira-kira 3 juta percobaan. Dengan 43.200 percobaan per hari dari satu alamat IP, itu berarti sekitar 69 hari. Namun bila penyerang memakai 100 alamat IP secara paralel, waktunya turun menjadi sekitar 17 jam.

> **Peringatan.** Contoh 6.2 memperlihatkan mengapa klaim seperti “token 32 bit dapat ditebak dalam hitungan jam” tidak dapat dinyatakan tanpa menyebut asumsinya. Waktu penebakan bergantung sepenuhnya pada laju percobaan yang diizinkan sistem. Perhitungan di atas bersifat ilustratif dan mengabaikan jeda jaringan serta deteksi anomali.

Pelajaran praktisnya ada dua. Pertama, gunakan entropi tinggi sehingga penebakan menjadi tidak layak berapa pun laju percobaannya. Kedua, jangan pernah bergantung pada kerahasiaan URL semata — karena URL memang beredar.

6.5.3 Batasan Keamanan QR

Kode QR mempercepat akses, tetapi QR **bukan** mekanisme pembuktian kehadiran fisik. Seseorang dapat memotret QR, menyalin URL, atau meneruskannya kepada orang lain selama token masih berlaku.

Untuk risiko rendah — pesanan makanan yang dibayar sebelum disajikan — pembatasan wewenang, TTL sesi, rotasi token, dan pencatatan aktivitas biasanya sudah memadai. Untuk risiko yang lebih tinggi, Anda dapat mempertimbangkan kontrol tambahan: konfirmasi nomor meja oleh pelanggan, kode dinamis pada layar meja, atau verifikasi oleh petugas.

Pilihan itu adalah keputusan desain, bukan keharusan teknis. Tambahkan kontrol hanya jika manfaat keamanannya sebanding dengan tambahan hambatan bagi pelanggan. Kontrol yang terlalu merepotkan akan dihindari staf, dan kontrol yang dihindari tidak melindungi apa pun.

6.6 Studi Kasus Terintegrasi: Pemesanan Tanpa Akun

Bagian ini menyatukan seluruh komponen dalam satu alur.

Langkah	Yang terjadi	Mekanisme yang bekerja
1. Meja dibuat	Pengelola kantin menambahkan meja dengan kode unik per kantin	`DiningTablePolicy`; batasan unik `(canteen_id, code)`
2. Token diterbitkan	Sistem membangkitkan token 256 bit dan menyimpan hash-nya	`QrTokenService::issue()` dalam transaksi berkunci
3. QR dicetak	QR menyandikan URL `/q/{token}` yang tidak menyebut kantin atau meja	Token mentah dikembalikan sekali, lalu tidak dapat diperoleh lagi
4. Pelanggan memindai	Permintaan melewati pembatas laju `qr-scan`	`throttle:qr-scan`, 30 permintaan per menit per IP
5. Token diselesaikan	Server memeriksa hash, status token, status meja, dan status kantin	`ResolveTableScan::resolve()`; kegagalan menghasilkan 404 generik
6. Sesi dibentuk	Sesi anonim terikat kantin dan meja, berlaku 4 jam	`CustomerSession` dengan kunci ULID
7. Token sesi dikirim	Token mentah disimpan pada cookie httpOnly, secure, SameSite lax	`ResolveTableQrController`
8. Pelanggan memesan	Setiap permintaan penting memvalidasi ulang sesi	Middleware validasi sesi; konteks meja dari sisi server
9. Akses berakhir	Sesi kedaluwarsa, ditutup, atau token QR dirotasi	`expires_at`, `status`, `QrTokenService::rotate()`

Tabel 6.7. Alur terintegrasi pemesanan tanpa akun.

Proyek pendamping menyediakan berkas uji `tests/Feature/QrTableSessionTest.php` yang membuktikan sebagian besar langkah tersebut. Dua metode uji berikut layak dibaca dengan saksama.

Kode 6.10 — tests/Feature/QrTableSessionTest.php (dua metode terpilih)

```php
public function test_rotate_revokes_old_token_and_keeps_single_active(): void
{
    $table = $this->table();
    $qr = app(QrTokenService::class);

    $plain1 = $qr->issue($table);
    $this->get(route('customer.scan', ['token' => $plain1]))->assertRedirect();

    $plain2 = $qr->rotate($table);

    // token lama gagal segera setelah rotasi
    $this->get(route('customer.scan', ['token' => $plain1]))->assertNotFound();
    // token baru sah
    $this->get(route('customer.scan', ['token' => $plain2]))->assertRedirect();

    $this->assertSame(1, TableQrToken::where('dining_table_id', $table->id)
        ->where('status', 'active')->count());
```

```
}

public function test_invalid_tokens_are_generic_404(): void
{
    // token acak
    $this->get(route('customer.scan', ['token' => 'tidak-ada-token']))
        ->assertNotFound();

    // ID numerik (anti-enumerasi)
    $this->get(route('customer.scan', ['token' => '1']))->assertNotFound();

    // token yang sudah dicabut
    $table = $this->table();
    $plain = app(QrTokenService::class)->issue($table);
    app(QrTokenService::class)->rotate($table);
    $this->get(route('customer.scan', ['token' => $plain]))->assertNotFound();

    // kantin nonaktif
    $inactiveCanteen = Canteen::factory()->create(['status' => 'inactive']);
    $t2 = $this->table($inactiveCanteen);
    $plain2 = app(QrTokenService::class)->issue($t2);
    $this->get(route('customer.scan', ['token' => $plain2]))->assertNotFound();
}
```

Metode uji kedua patut diperhatikan. Ia menguji empat penyebab kegagalan yang berbeda dan mengharapkan **respons yang sama persis** untuk keempatnya. Pengujian semacam ini jarang ditulis pemula, karena secara fungsional keempat kasus itu "sudah gagal". Justru keseragamannya yang diuji di sini: perbedaan sekecil apa pun antara keempat respons akan menjadi saluran informasi bagi penyerang.

Jalankan berkas uji tersebut di terminal dengan perintah berikut.

```
Kode 6.11 — Menjalankan pengujian QR dan sesi (di terminal)
php artisan test --filter=QrTableSessionTest
```

Studi kasus ini menunjukkan prinsip utamanya: aplikasi tidak perlu mengetahui identitas pelanggan untuk mengelola pesanan meja, tetapi aplikasi tetap harus menjaga konteks meja, masa hidup akses, dan batas wewenang secara konsisten.

Rangkuman Bab

- Sesi anonim mengikat aktivitas pada konteks layanan — meja dan kantin — bukan pada identitas pengguna.
- Token QR dan token sesi memiliki fungsi berbeda: token QR membuka titik masuk, token sesi mempertahankan akses setelah masuk.
- Token *opaque* adalah untai acak yang maknanya diperoleh melalui *lookup* sisi server, sehingga mudah dicabut seketika.
- Token akses dibangkitkan dari sumber acak kriptografis. Proyek pendamping memakai 32 bita acak yang dikodekan base64url, yaitu entropi 256 bit.
- Entropi ditentukan oleh sumber acak, bukan oleh panjang teks hasil pengodean; Persamaan 6.1 memberi batas atas ketika pengodean tidak sejajar dengan batas bit.
- Server hanya menyimpan *hash* token, sehingga kebocoran basis data tidak menyerahkan kredensial yang dapat langsung dipakai.

- *Signed URL* melindungi integritas parameter, bukan kerahasiaan token — dan sulit dipadukan dengan QR cetak permanen.
- URL titik masuk tidak memuat pengenal kantin atau meja; konteks selalu ditentukan sisi server dari token.
- Seluruh kegagalan penelusuran token menghasilkan respons yang sama agar tidak membocorkan keberadaan sumber daya.
- *Cookie* sesi memakai `httpOnly`, `secure`, dan `SameSite=lax`; namun kedaluwarsa yang mengikat tetap yang tersimpan di basis data.
- Setiap pesanan dikaitkan dengan meja melalui sesi yang divalidasi sisi server, bukan melalui nomor meja dari masukan pengguna.

Bab berikutnya melanjutkan ke katalog menu yang dilihat pelanggan melalui sesi anonim ini: pengelolaan menu tenant, *modifier*, stok, dan kueri katalog publik yang harus melintasi banyak tenant dalam satu kantin tanpa membocorkan data.

Istilah Penting

Istilah	Makna ringkas
Sesi anonim	Sesi tanpa identitas pengguna, terikat pada konteks layanan
Token QR	Token yang disandikan QR untuk menunjukkan titik masuk meja
Token sesi	Token yang membuktikan adanya sesi anonim aktif pada klien
Token opaque	Token acak tanpa makna bisnis, divalidasi melalui lookup
JWT	Token yang membawa klaim dan diverifikasi melalui tanda tangan
Entropi	Ukuran ketakterdugaan token, dinyatakan dalam bit
CSPRNG	Sumber acak yang aman untuk keperluan kriptografis
Base64url	Pengodean base64 yang aman disisipkan ke dalam URL
TTL	Batas masa hidup token atau sesi (time to live)
Rotasi token	Penerbitan token baru yang sekaligus mencabut token lama
Signed URL	URL bertanda tangan yang memungkinkan deteksi manipulasi parameter
Rate limiting	Pembatasan jumlah permintaan dalam periode tertentu
Bearer credential	Kredensial yang dapat dipakai siapa pun yang memegangnya
Enumerasi	Penelusuran sumber daya dengan mencoba pengenal berurutan
ULID	Pengenal unik terurut waktu yang tidak berurutan secara tebakan

Tabel 6.8. Istilah penting pada Bab 6.

Pertanyaan dan Latihan

A. Pemahaman

1. Jelaskan perbedaan sesi terautentikasi dan sesi anonim. Mengapa sesi anonim tidak boleh diberi wewenang seperti akun operator tenant?
2. Bedakan fungsi token QR dan token sesi dalam alur pemesanan tanpa akun, termasuk perbedaan masa hidup dan cara pencabutannya.

3. Jelaskan mengapa token *opaque* umumnya lebih mudah dicabut daripada JWT.
4. Mengapa server menyimpan *hash* token, bukan token mentah? Apa yang tetap dapat dilakukan penyerang yang memperoleh salinan basis data?
5. Mengapa SHA-256 memadai untuk token acak, sedangkan kata sandi memerlukan algoritme *hash* yang sengaja dibuat lambat?

B. Penerapan

6. Hitung entropi token 40 karakter alfanumerik apabila setiap karakter dipilih dari 62 simbol. Gunakan Persamaan 6.1. Apakah hasilnya melebihi 128 bit?
7. Dengan asumsi terdapat 5.000 token aktif, hitung peluang satu percobaan menebak token yang sah untuk token 96 bit dan 256 bit. Gunakan Persamaan 6.2.
8. Rancang pekerjaan terjadwal untuk menandai sesi kedaluwarsa sebagai `expired` dan menghapus baris yang sudah lama. Tuliskan kriterianya, lalu jelaskan mengapa validasi `expires_at` tetap diperlukan pada *middleware*.
9. Tulis metode uji yang membuktikan bahwa sesi pelanggan tidak dapat dipakai setelah `expires_at` terlampaui, tanpa menunggu empat jam nyata.
10. Ubah `QrTokenService::issue()` agar menerima masa berlaku token dan tuliskan satu skenario operasional ketika `expires_at` pada token QR berguna.

C. Analisis dan Desain

11. Sebuah kantin mencetak QR permanen pada setiap meja dan tidak dapat mengganti stikernya setiap minggu. Bandingkan dua pilihan desain: (a) QR berisi token acak yang harus dirotasi dan dicetak ulang; (b) QR berisi pengenal meja yang stabil dengan kontrol tambahan di server. Tentukan rekomendasi Anda beserta manfaat, risiko, dan konsekuensi operasionalnya.
12. Proyek pendamping tidak memakai *signed URL* pada rute pemindaian. Uraikan argumen yang mendukung keputusan itu, lalu uraikan satu kondisi ketika *signed URL* justru akan bermanfaat.
13. Seorang pelanggan memotret QR sebuah meja lalu membagikan fotonya kepada temannya di luar kantin. Identifikasi risiko yang mungkin terjadi dan rancang sedikitnya empat lapisan pengendalian yang proporsional.
14. Analisis dampaknya apabila `ResolveTableScan::resolve()` mengembalikan pesan galat yang berbeda untuk token tidak ditemukan, token dicabut, dan kantin nonaktif. Jelaskan informasi apa yang dapat disimpulkan penyerang dari perbedaan itu.
15. Rancang mekanisme identifikasi anonim untuk layanan peminjaman loker kampus tanpa akun. Tentukan konteks yang diikat sesi, jenis token, masa berlaku, cara pencabutan, batas wewenang, dan risiko penyalahgunaan yang perlu ditangani.

Daftar Pustaka

Buku

Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems* (3rd ed.). Wiley.

Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

Standar dan Spesifikasi

Barker, E., & Kelsey, J. (2015). *Recommendation for Random Number Generation Using Deterministic Random Bit Generators* (NIST Special Publication 800-90A Rev. 1). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-90Ar1

Josefsson, S. (2006). *The Base16, Base32, and Base64 Data Encodings* (RFC 4648). Internet Engineering Task Force. https://doi.org/10.17487/RFC4648

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

International Organization for Standardization. (2015). *ISO/IEC 18004:2015 — Automatic identification and data capture techniques: QR Code bar code symbology specification*.

Dokumentasi dan Sumber Daring

Laravel LLC. (2026). *Rate Limiting, URL Generation, HTTP Session, Middleware — Laravel 13.x Documentation*. Diakses dari https://laravel.com/docs/13.x/rate-limiting

OWASP Foundation. (2021). *OWASP Session Management Cheat Sheet*. Diakses dari https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

PHP Group. (2026). *random_bytes — PHP Manual*. Diakses dari https://www.php.net/manual/en/function.random-bytes.php

[PERLU VERIFIKASI: metadata, URL, tanggal akses, edisi, dan DOI seluruh rujukan daring dan standar]

Sumber Proyek (Dokumen Internal dan Repositori)

Suwardiyanto, D. (2026). *Kantin Multi-Tenant — Repositori Kode Pendamping*, tag `pertemuan-06`. https://github.com/devitsuwardiyanto/kantin-multi-tenant

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 — Modul 6: Meja, QR, Token Opaque, dan Sesi Pelanggan Anonim*. Dokumen praktikum.

Catatan: seluruh data numerik dan perhitungan pada bab ini bersifat ilustratif untuk keperluan pembelajaran. Potongan kode yang menyebutkan nama berkas telah diselaraskan dengan repositori pendamping pada tag pertemuan-06; contoh yang ditandai "contoh generik" disajikan untuk menjelaskan mekanisme dan tidak berasal dari repositori.